

Studentu rezultatu sistema

v2.0

Generated by Doxygen 1.17.0

1 Directory Hierarchy	1
1.1 Directories	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Data Structure Index	5
3.1 Data Structures	5
4 File Index	7
4.1 File List	7
5 Directory Documentation	9
5.1 ConsoleApplication1 Directory Reference	9
6 Data Structure Documentation	11
6.1 DuomenuKlaida Struct Reference	11
6.1.1 Detailed Description	11
6.1.2 Constructor & Destructor Documentation	11
6.1.2.1 DuomenuKlaida()	11
6.2 FailoKlaida Struct Reference	12
6.2.1 Detailed Description	12
6.2.2 Constructor & Destructor Documentation	12
6.2.2.1 FailoKlaida()	12
6.3 Studentas Class Reference	12
6.3.1 Detailed Description	13
6.3.2 Constructor & Destructor Documentation	13
6.3.2.1 Studentas() [1/3]	13
6.3.2.2 ~Studentas()	13
6.3.2.3 Studentas() [2/3]	13
6.3.2.4 Studentas() [3/3]	14
6.3.3 Member Function Documentation	14
6.3.3.1 operator=() [1/2]	14
6.3.3.2 operator=() [2/2]	14
6.3.4 Field Documentation	14
6.3.4.1 egzaminas	14
6.3.4.2 n	14
6.3.4.3 nd	14
6.4 Zmogus Class Reference	15
6.4.1 Detailed Description	15
6.4.2 Constructor & Destructor Documentation	15
6.4.2.1 Zmogus()	15
6.4.2.2 ~Zmogus()	15
6.4.3 Field Documentation	15

6.4.3.1 pavarde	15
6.4.3.2 vardas	15
7 File Documentation	17
7.1 ConsoleApplication1/exceptions.h File Reference	17
7.2 exceptions.h	17
7.3 ConsoleApplication1/io.cpp File Reference	18
7.3.1 Function Documentation	18
7.3.1.1 ivesitiRankiniu()	18
7.3.1.2 nuskaitytiStudentus()	18
7.3.1.3 skaitytiSveika()	18
7.3.1.4 spausdintiStream()	19
7.3.1.5 spausdintiRezultatus()	19
7.4 io.cpp	19
7.5 ConsoleApplication1/io.h File Reference	21
7.5.1 Function Documentation	22
7.5.1.1 ivesitiRankiniu()	22
7.5.1.2 nuskaitytiStudentus()	22
7.5.1.3 skaitytiSveika()	22
7.5.1.4 spausdintiRezultatus()	22
7.6 io.h	22
7.7 ConsoleApplication1/main.cpp File Reference	23
7.7.1 Function Documentation	23
7.7.1.1 main()	23
7.8 main.cpp	23
7.9 ConsoleApplication1/skaiciavimas.cpp File Reference	25
7.9.1 Function Documentation	25
7.9.1.1 skaiciuotiGalutini()	25
7.9.1.2 skaiciuotiMediana()	26
7.9.1.3 skaiciuotiVidurki()	26
7.10 skaiciavimas.cpp	26
7.11 ConsoleApplication1/skaiciavimas.h File Reference	26
7.11.1 Function Documentation	27
7.11.1.1 skaiciuotiGalutini()	27
7.11.1.2 skaiciuotiMediana()	27
7.11.1.3 skaiciuotiVidurki()	27
7.12 skaiciavimas.h	27
7.13 ConsoleApplication1/studentas.cpp File Reference	27
7.13.1 Function Documentation	28
7.13.1.1 operator<<()	28
7.13.1.2 operator>>()	28
7.14 studentas.cpp	28

7.15 ConsoleApplication1/studentas.h File Reference	29
7.15.1 Function Documentation	30
7.15.1.1 operator<<()	30
7.15.1.2 operator>>()	30
7.16 studentas.h	30
7.17 ConsoleApplication1/studentas_utils.cpp File Reference	30
7.17.1 Function Documentation	31
7.17.1.1 generuotiPazymius()	31
7.17.1.2 generuotiVarda()	31
7.17.1.3 pasirinktiRusiavima()	31
7.17.1.4 rusiuotiStudentus()	31
7.18 studentas_utils.cpp	32
7.19 ConsoleApplication1/studentas_utils.h File Reference	33
7.19.1 Function Documentation	33
7.19.1.1 generuotiPazymius()	33
7.19.1.2 generuotiVarda()	33
7.19.1.3 pasirinktiRusiavima()	33
7.19.1.4 rusiuotiStudentus()	33
7.20 studentas_utils.h	34
7.21 ConsoleApplication1/testavimas.cpp File Reference	34
7.21.1 Function Documentation	34
7.21.1.1 generuotiFaila()	34
7.21.1.2 isvestiKategorijaFaila()	35
7.21.1.3 nuskaitytilsFailo()	35
7.21.1.4 testuotiKlase()	35
7.21.1.5 tyrimas1_failuKurimas()	35
7.21.1.6 tyrimasKonteineriu()	35
7.21.1.7 tyrimasStrategiju()	35
7.22 testavimas.cpp	36
7.23 ConsoleApplication1/testavimas.h File Reference	41
7.23.1 Function Documentation	42
7.23.1.1 apskaiciuotiGalutiniBala()	42
7.23.1.2 benchmarkKonteineris()	42
7.23.1.3 benchmarkStrategijos()	43
7.23.1.4 generuotiFaila()	43
7.23.1.5 isvestiKategorijaFaila()	43
7.23.1.6 nuskaitytilsFailo()	43
7.23.1.7 nuskaitytilsFailoT()	43
7.23.1.8 rusiuotiPagalGalutini()	44
7.23.1.9 strategija1()	44
7.23.1.10 strategija2()	44
7.23.1.11 strategija3()	44

7.23.1.12 testuotiKlase()	44
7.23.1.13 tyrimas1_failuKurimas()	44
7.23.1.14 tyrimasKonteineriu()	45
7.23.1.15 tyrimasStrategiju()	45
7.24 testavimas.h	45
7.25 ConsoleApplication1/zmogus.cpp File Reference	48
7.26 zmogus.cpp	48
7.27 ConsoleApplication1/zmogus.h File Reference	48
7.28 zmogus.h	48
Index	49

Chapter 1

Directory Hierarchy

1.1 Directories

ConsoleApplication1	9
exceptions.h	17
io.cpp	18
io.h	21
main.cpp	23
skaiciavimas.cpp	25
skaiciavimas.h	26
studentas.cpp	27
studentas.h	29
studentas_utils.cpp	30
studentas_utils.h	33
testavimas.cpp	34
testavimas.h	41
zmogus.cpp	48
zmogus.h	48

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

std::runtime_error	
DuomenuKlaida	11
FailoKlaida	12
Zmogus	15
Studentas	12

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

DuomenuKlaida	11
FailoKlaida	12
Studentas	
Studento duomenu klase, paveldinti is Zmogus	12
Zmogus	15

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

ConsoleApplication1/exceptions.h	17
ConsoleApplication1/io.cpp	18
ConsoleApplication1/io.h	21
ConsoleApplication1/main.cpp	23
ConsoleApplication1/skaiciavimas.cpp	25
ConsoleApplication1/skaiciavimas.h	26
ConsoleApplication1/studentas.cpp	27
ConsoleApplication1/studentas.h	29
ConsoleApplication1/studentas_utils.cpp	30
ConsoleApplication1/studentas_utils.h	33
ConsoleApplication1/testavimas.cpp	34
ConsoleApplication1/testavimas.h	41
ConsoleApplication1/zmogus.cpp	48
ConsoleApplication1/zmogus.h	48

Chapter 5

Directory Documentation

5.1 ConsoleApplication1 Directory Reference

Files

- file [exceptions.h](#)
- file [io.cpp](#)
- file [io.h](#)
- file [main.cpp](#)
- file [skaiciavimas.cpp](#)
- file [skaiciavimas.h](#)
- file [studentas.cpp](#)
- file [studentas.h](#)
- file [studentas_utils.cpp](#)
- file [studentas_utils.h](#)
- file [testavimas.cpp](#)
- file [testavimas.h](#)
- file [zmogus.cpp](#)
- file [zmogus.h](#)

Chapter 6

Data Structure Documentation

6.1 DuomenuKlaida Struct Reference

```
#include <exceptions.h>
```

Inheritance diagram for DuomenuKlaida:

Collaboration diagram for DuomenuKlaida:

Public Member Functions

- [DuomenuKlaida](#) (const std::string &msg)

6.1.1 Detailed Description

Definition at line 12 of file [exceptions.h](#).

6.1.2 Constructor & Destructor Documentation

6.1.2.1 DuomenuKlaida()

```
DuomenuKlaida::DuomenuKlaida (  
    const std::string & msg) [inline], [explicit]
```

Definition at line 14 of file [exceptions.h](#).

The documentation for this struct was generated from the following file:

- ConsoleApplication1/[exceptions.h](#)

6.2 FailoKlaida Struct Reference

```
#include <exceptions.h>
```

Inheritance diagram for FailoKlaida:

Collaboration diagram for FailoKlaida:

Public Member Functions

- [FailoKlaida](#) (const std::string &msg)

6.2.1 Detailed Description

Definition at line 7 of file [exceptions.h](#).

6.2.2 Constructor & Destructor Documentation

6.2.2.1 FailoKlaida()

```
FailoKlaida::FailoKlaida (
    const std::string & msg) [inline], [explicit]
```

Definition at line 9 of file [exceptions.h](#).

The documentation for this struct was generated from the following file:

- ConsoleApplication1/[exceptions.h](#)

6.3 Studentas Class Reference

Studento duomenų klase, paveldinti iš [Zmogus](#).

```
#include <studentas.h>
```

Inheritance diagram for Studentas:

Collaboration diagram for Studentas:

Public Member Functions

- [Studentas](#) ()
Default konstruktorius. Visi laukai nustatomi į 0/"".
- [~Studentas](#) () noexcept override
- [Studentas](#) (const [Studentas](#) &kitas)
Kopijavimo konstruktorius.
- [Studentas](#) & [operator=](#) (const [Studentas](#) &kitas)
- [Studentas](#) ([Studentas](#) &&kitas) noexcept
Perkelimo konstruktorius. Saltinis paliekamas tuscias.
- [Studentas](#) & [operator=](#) ([Studentas](#) &&kitas) noexcept

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()
- virtual [~Zmogus](#) ()=0

Data Fields

- `std::vector< int >` [nd](#)
- `int` [n](#)
- `int` [egzaminas](#)

Data Fields inherited from [Zmogus](#)

- `std::string` [vardas](#)
- `std::string` [pavarde](#)

6.3.1 Detailed Description

Studento duomenų klase, paveldinti iš [Zmogus](#).

Realizuoja 5-iu metodu taisyklę (Rule of Five): kopijavimo/perkelimo konstruktorius, kopijavimo/perkelimo priskyrimo operatorius ir destruktorius.

Definition at line 14 of file [studentas.h](#).

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Studentas() [1/3]

```
Studentas::Studentas ()
```

Default konstruktorius. Visi laukai nustatomi į 0/"".

Definition at line 4 of file [studentas.cpp](#).

6.3.2.2 ~Studentas()

```
Studentas::~~Studentas () [override], [noexcept]
```

Definition at line 9 of file [studentas.cpp](#).

6.3.2.3 Studentas() [2/3]

```
Studentas::Studentas (  
    const Studentas & kitas)
```

Kopijavimo konstruktorius.

Definition at line 15 of file [studentas.cpp](#).

6.3.2.4 Studentas() [3/3]

```
Studentas::Studentas (
    Studentas && kitas) [noexcept]
```

Perkelimo konstruktorius. Salinis paliekamas tuscias.

Definition at line 37 of file [studentas.cpp](#).

6.3.3 Member Function Documentation

6.3.3.1 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & kitas)
```

Definition at line 23 of file [studentas.cpp](#).

6.3.3.2 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && kitas) [noexcept]
```

Definition at line 49 of file [studentas.cpp](#).

6.3.4 Field Documentation

6.3.4.1 egzaminas

```
int Studentas::egzaminas
```

Definition at line 19 of file [studentas.h](#).

6.3.4.2 n

```
int Studentas::n
```

Definition at line 18 of file [studentas.h](#).

6.3.4.3 nd

```
std::vector<int> Studentas::nd
```

Definition at line 17 of file [studentas.h](#).

The documentation for this class was generated from the following files:

- ConsoleApplication1/[studentas.h](#)
- ConsoleApplication1/[studentas.cpp](#)

6.4 Zmogus Class Reference

```
#include <zmogus.h>
```

Inheritance diagram for Zmogus:

Collaboration diagram for Zmogus:

Public Member Functions

- [Zmogus](#) ()
- virtual [~Zmogus](#) ()=0

Data Fields

- std::string [vardas](#)
- std::string [pavarde](#)

6.4.1 Detailed Description

Definition at line 7 of file [zmogus.h](#).

6.4.2 Constructor & Destructor Documentation

6.4.2.1 Zmogus()

```
Zmogus::Zmogus ()
```

Definition at line 3 of file [zmogus.cpp](#).

6.4.2.2 ~Zmogus()

```
Zmogus::~Zmogus () [pure virtual]
```

Definition at line 7 of file [zmogus.cpp](#).

6.4.3 Field Documentation

6.4.3.1 pavarde

```
std::string Zmogus::pavarde
```

Definition at line 11 of file [zmogus.h](#).

6.4.3.2 vardas

```
std::string Zmogus::vardas
```

Definition at line 10 of file [zmogus.h](#).

The documentation for this class was generated from the following files:

- ConsoleApplication1/[zmogus.h](#)
- ConsoleApplication1/[zmogus.cpp](#)

Chapter 7

File Documentation

7.1 ConsoleApplication1/exceptions.h File Reference

```
#include <stdexcept>
#include <string>
```

Include dependency graph for exceptions.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [FailoKlaida](#)
- struct [DuomenuKlaida](#)

7.2 exceptions.h

[Go to the documentation of this file.](#)

```
00001 #ifndef EXCEPTIONS_H
00002 #define EXCEPTIONS_H
00003
00004 #include <stdexcept>
00005 #include <string>
00006
00007 struct FailoKlaida : public std::runtime_error
00008 {
00009     explicit FailoKlaida( const std::string& msg ) : std::runtime_error( msg ) {}
00010 };
00011
00012 struct DuomenuKlaida : public std::runtime_error
00013 {
00014     explicit DuomenuKlaida( const std::string& msg ) : std::runtime_error( msg ) {}
00015 };
00016
00017 #endif
```

7.3 ConsoleApplication1/io.cpp File Reference

```
#include "io.h"
#include "studentas_utils.h"
#include "skaiciavimas.h"
#include "exceptions.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <iomanip>
#include <limits>
#include <chrono>
```

Include dependency graph for io.cpp:

Functions

- bool [skaitytiSveika](#) (int &reiksme, int min_val, int max_val)
- template<typename Stream>
void [spausdintiStream](#) (Stream &out, const std::vector< [Studentas](#) > &studentai, bool mediana)
- void [spausdintiRezultatus](#) (std::vector< [Studentas](#) > &studentai, int m, bool mediana)
- std::vector< [Studentas](#) > [investiRankiniu](#) (int &m, int &n)
- std::vector< [Studentas](#) > [nuskaitytiStudentus](#) ()

7.3.1 Function Documentation

7.3.1.1 [investiRankiniu\(\)](#)

```
std::vector< Studentas > investiRankiniu (  
    int & m,  
    int & n)
```

Definition at line 91 of file [io.cpp](#).

7.3.1.2 [nuskaitytiStudentus\(\)](#)

```
std::vector< Studentas > nuskaitytiStudentus ()
```

Definition at line 142 of file [io.cpp](#).

7.3.1.3 [skaitytiSveika\(\)](#)

```
bool skaitytiSveika (  
    int & reiksme,  
    int min_val,  
    int max_val)
```

Definition at line 12 of file [io.cpp](#).

7.3.1.4 spausdintiStream()

```
template<typename Stream>
void spausdintiStream (
    Stream & out,
    const std::vector< Studentas > & studentai,
    bool mediana)
```

Definition at line 28 of file [io.cpp](#).

7.3.1.5 spausdintiRezultatus()

```
void spausdintiRezultatus (
    std::vector< Studentas > & studentai,
    int m,
    bool mediana)
```

Definition at line 62 of file [io.cpp](#).

7.4 io.cpp

[Go to the documentation of this file.](#)

```
00001 #include "io.h"
00002 #include "studentas_utils.h"
00003 #include "skaiciavimas.h"
00004 #include "exceptions.h"
00005 #include <iostream>
00006 #include <fstream>
00007 #include <sstream>
00008 #include <iomanip>
00009 #include <limits>
00010 #include <chrono>
00011
00012 bool skaitytiSveika( int& reiksme, int min_val, int max_val )
00013 {
00014     if ( !( std::cin » reiksme ) )
00015     {
00016         std::cin.clear( );
00017         std::cin.ignore( std::numeric_limits<std::streamsize>::max( ), '\n' );
00018         return false;
00019     }
00020
00021     if ( reiksme < min_val || reiksme > max_val )
00022         return false;
00023
00024     return true;
00025 }
00026
00027 template <typename Stream>
00028 void spausdintiStream( Stream& out, const std::vector<Studentas>& studentai, bool mediana )
00029 {
00030     out « "\n" « std::string( 70, '-' ) « "\n";
00031
00032     out « std::left « std::setw( 15 ) « "Pavarde"
00033         « std::setw( 15 ) « "Vardas"
00034         « std::setw( 20 ) « "Galutinis (Vid.)";
00035
00036     if ( mediana ) out « std::setw( 20 ) « "Galutinis (Med.)";
00037
00038     out « "\n" « std::string( 70, '-' ) « "\n";
00039
00040     for ( const auto& s : studentai )
00041     {
00042         double vid = skaiciuotiGalutini( skaiciuotiVidurki( s.nd, s.n ), s.egzaminas );
00043
00044         out « std::left « std::setw( 15 ) « s.pavarde
00045             « std::setw( 15 ) « s.vardas
00046             « std::fixed « std::setprecision( 2 )
00047             « std::setw( 20 ) « vid;
```

```

00048
00049     if ( mediana )
00050     {
00051         double med = skaiciuotiGalutini( skaiciuotiMediana( s.nd, s.n ), s.egzaminas );
00052         out « std::setw( 20 ) « med;
00053     }
00054
00055     out « "\n";
00056 }
00057
00058 out « std::string( 70, '-' ) « "\n";
00059 }
00060
00061
00062 void spausdintiRezultatus( std::vector<Studentas>& studentai, int m, bool mediana )
00063 {
00064     int rusiavimas = pasirinktiRusiavima( );
00065     rusiuotiStudentus( studentai, rusiavimas );
00066
00067     int outputPasirinkimas;
00068     std::cout « "\nIsvedimo vieta:\n 1 - Ekranas\n 2 - Failas\nPasirinkimas: ";
00069
00070     while ( !skaitytiSveika( outputPasirinkimas, 1, 2 ) ) {
00071         std::cout « "Neteisinga reiksme. Pasirinkite 1 arba 2: ";
00072     }
00073
00074     if ( outputPasirinkimas == 1 )
00075     {
00076         spausdintiIStream( std::cout, studentai, mediana );
00077     }
00078     else
00079     {
00080         std::ofstream stream( "rezultatai.txt" );
00081         if ( !stream.is_open( ) )
00082         {
00083             throw FailoKlaida( "Nepavyko atidaryti failo rezultatai.txt rasymui." );
00084         }
00085
00086         spausdintiIStream( stream, studentai, mediana );
00087         std::cout « "Rezultatai issaugoti faile: rezultatai.txt\n";
00088     }
00089 }
00090
00091 std::vector<Studentas> ivesitiRankiniu( int& m, int& n )
00092 {
00093     std::cout « "Kiek namu darbu? ";
00094
00095     while ( !skaitytiSveika( n, 1, 100 ) ) {
00096         std::cout « "Neteisinga reiksme. Iveskite 1-100: ";
00097     }
00098
00099     m = 0;
00100     std::vector<Studentas> studentai;
00101
00102     while ( true )
00103     {
00104         Studentas studentas;
00105
00106         std::cout « "\n--- Studentas #" « ( m + 1 ) « " (arba iveskite 'baigti') ---\n";
00107         std::cout « "Vardas: ";
00108
00109         std::cin » studentas.vardas;
00110
00111         if ( studentas.vardas == "baigti" )
00112             break;
00113
00114         std::cout « "Pavarde: ";
00115         std::cin » studentas.pavarde;
00116
00117         studentas.n = n;
00118         studentas.nd.resize( n );
00119
00120         for ( int j = 0; j < n; j++ )
00121         {
00122             std::cout « " ND" « ( j + 1 ) « " balas (1-10): ";
00123             while ( !skaitytiSveika( studentas.nd [ j ], 1, 10 ) ) {
00124                 std::cout « " Neteisinga reiksme (1-10): ";
00125             }
00126         }
00127
00128         std::cout « "Egzamino balas (1-10): ";
00129
00130         while ( !skaitytiSveika( studentas.egzaminas, 1, 10 ) ) {
00131             std::cout « " Neteisinga reiksme (1-10): ";
00132         }
00133
00134         studentai.push_back( studentas );

```

```

00135
00136         m++;
00137     }
00138
00139     return studentai;
00140 }
00141
00142 std::vector<Studentas> nuskaitytiStudentus( )
00143 {
00144     std::ifstream stream( "kursiokai.txt" );
00145     if ( !stream.is_open( ) )
00146         throw FailoKlaida( "Nepavyko atidaryti failo kursiokai.txt." );
00147
00148     auto start = std::chrono::high_resolution_clock::now( );
00149
00150     std::vector<Studentas> out;
00151
00152     std::string line;
00153     if ( !std::getline( stream, line ) )
00154         return out;
00155
00156     std::stringstream header_stream( line );
00157
00158     std::string column;
00159     std::vector<std::string> cols;
00160
00161     while ( header_stream » column )
00162         cols.push_back( column );
00163
00164     size_t nd_count = cols.size( ) >= 3 ? cols.size( ) - 3 : 0;
00165
00166     while ( std::getline( stream, line ) )
00167     {
00168         if ( line.empty( ) )
00169             continue;
00170
00171         std::stringstream ss( line );
00172
00173         Studentas studentas;
00174         studentas.n = static_cast<int>( nd_count );
00175         studentas.nd.resize( nd_count );
00176
00177         if ( !( ss » studentas.vardas » studentas.pavarde ) )
00178             throw DuomenuKlaida( "Nepavyko nuskaityti vardo arba pavardes: " + line );
00179
00180         for ( size_t i = 0; i < nd_count; i++ )
00181         {
00182             if ( !( ss » studentas.nd [ i ] ) )
00183                 throw DuomenuKlaida( "Nepavyko nuskaityti namu darbu pazymio eiluteje: " + line );
00184         }
00185
00186         if ( !( ss » studentas.egzaminas ) )
00187             throw DuomenuKlaida( "Nepavyko nuskaityti egzamino balo eiluteje: " + line );
00188
00189         out.push_back( studentas );
00190     }
00191
00192     auto end = std::chrono::high_resolution_clock::now( );
00193     auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>( end - start );
00194     std::cout « "Nuskaityti " « out.size( ) « " studentu is per " « std::fixed « std::setprecision( 3
00195 ) « elapsed.count( ) « " ms\n";
00196
00197     return out;
00198 }

```

7.5 ConsoleApplication1/io.h File Reference

```

#include "studentas.h"
#include <vector>
#include <iostream>

```

Include dependency graph for io.h: This graph shows which files directly or indirectly include this file:

Functions

- bool [skaitytiSveika](#) (int &reiksme, int min_val, int max_val)

- void `spausdintiRezultatus` (std::vector< `Studentas` > &studentai, int m, bool mediana)
- std::vector< `Studentas` > `investiRankiniu` (int &m, int &n)
- std::vector< `Studentas` > `nuskaitytiStudentus` ()

7.5.1 Function Documentation

7.5.1.1 investiRankiniu()

```
std::vector< Studentas > investiRankiniu (
    int & m,
    int & n)
```

Definition at line 91 of file [io.cpp](#).

7.5.1.2 nuskaitytiStudentus()

```
std::vector< Studentas > nuskaitytiStudentus ()
```

Definition at line 142 of file [io.cpp](#).

7.5.1.3 skaitytiSveika()

```
bool skaitytiSveika (
    int & reiksme,
    int min_val,
    int max_val)
```

Definition at line 12 of file [io.cpp](#).

7.5.1.4 spausdintiRezultatus()

```
void spausdintiRezultatus (
    std::vector< Studentas > & studentai,
    int m,
    bool mediana)
```

Definition at line 62 of file [io.cpp](#).

7.6 io.h

[Go to the documentation of this file.](#)

```
00001 #ifndef IO_H
00002 #define IO_H
00003
00004 #include "studentas.h"
00005 #include <vector>
00006 #include <iostream>
00007
00008 bool skaitytiSveika( int& reiksme, int min_val, int max_val );
00009
00010 void spausdintiRezultatus( std::vector<Studentas>& studentai, int m, bool mediana );
00011
00012 std::vector<Studentas> investiRankiniu( int& m, int& n );
00013 std::vector<Studentas> nuskaitytiStudentus( );
00014
00015 #endif
```

7.7 ConsoleApplication1/main.cpp File Reference

```
#include "io.h"
#include "studentas_utils.h"
#include "exceptions.h"
#include "testavimas.h"
#include <iostream>
#include <cstdlib>
#include <ctime>
```

Include dependency graph for main.cpp:

Functions

- `int main()`

7.7.1 Function Documentation

7.7.1.1 main()

```
int main ()
```

Definition at line 10 of file [main.cpp](#).

7.8 main.cpp

[Go to the documentation of this file.](#)

```
00001 #include "io.h"
00002 #include "studentas_utils.h"
00003 #include "exceptions.h"
00004 #include "testavimas.h"
00005 #include <iostream>
00006 #include <cstdlib>
00007 #include <ctime>
00008
00009
00010 int main( )
00011 {
00012     std::srand( static_cast<unsigned>( std::time( nullptr ) ) );
00013
00014     int skaiciavimas = { };
00015     std::cout << "Galutinio balo skaiciavimas:\n 1 - Vidurkis\n 2 - Mediana\nPasirinkimas: ";
00016
00017     while ( !skaitytiSveika( skaiciavimas, 1, 2 ) ) {
00018         std::cout << "Neteisinga reiksme. Pasirinkite 1 arba 2: ";
00019     }
00020
00021     bool mediana = ( skaiciavimas == 2 );
00022
00023     int m { };
00024     int n { };
00025     int meniu;
00026
00027     do {
00028         std::cout << "\n===== MENU =====\n";
00029         std::cout << " 1 - Ivesti duomenis rankiniu budu\n";
00030         std::cout << " 2 - Generuoti tik pazymius\n";
00031         std::cout << " 3 - Generuoti vardus, pavardes ir pazymius\n";
00032         std::cout << " 4 - Nuskaityti studentus is failo\n";
00033         std::cout << " 5 - 1 tyrimas: Failu generavimas\n";
00034         std::cout << " 6 - 2 tyrimas: Konteineriu palyginimas (vector, list, deque)\n";
00035         std::cout << " 7 - 3 tyrimas: Strategiju palyginimas (1, 2, 3 strategijos)\n";
00036         std::cout << " 8 - Testuoti Studentas klase (Rule of Five + I/O operatoriai)\n";
```

```

00037     std::cout << " 9 - Baigti darba\n";
00038     std::cout << "Pasirinkimas: ";
00039
00040     if ( !skaitytiSveika( meniu, 1, 9 ) )
00041     {
00042         std::cout << "Neteisinga reiksme.\n";
00043         continue;
00044     }
00045
00046     try
00047     {
00048         switch ( meniu )
00049         {
00050             case 1:
00051             {
00052                 std::vector<Studentas> studentai = iverstiRankiniu( m, n );
00053
00054                 if ( m > 0 )
00055                     spausdintiRezultatus( studentai, m, mediana );
00056                 else
00057                     std::cout << "Nera studentu duomenu.\n";
00058                 break;
00059             }
00060
00061             case 2:
00062             {
00063                 std::cout << "Studentu skaicius: ";
00064                 while ( !skaitytiSveika( m, 1, 10000 ) ) {
00065                     std::cout << "Neteisinga reiksme: ";
00066                 }
00067
00068                 std::cout << "Namu darbu skaicius: ";
00069                 while ( !skaitytiSveika( n, 1, 100 ) ) {
00070                     std::cout << "Neteisinga reiksme: ";
00071                 }
00072
00073                 std::vector<Studentas> studentai( m );
00074
00075                 for ( int i = 0; i < m; i++ )
00076                 {
00077                     std::cout << " Studentas #" << ( i + 1 ) << " vardas: ";
00078                     std::cin >> studentai[i].vardas;
00079
00080                     std::cout << " Studentas #" << ( i + 1 ) << " pavarde: ";
00081                     std::cin >> studentai[i].pavarde;
00082
00083                     generuotiPazymius( studentai[i], n );
00084                 }
00085
00086                 spausdintiRezultatus( studentai, m, mediana );
00087                 break;
00088             }
00089
00090             case 3:
00091             {
00092                 std::cout << "Studentu skaicius: ";
00093                 while ( !skaitytiSveika( m, 1, 10000 ) ) {
00094                     std::cout << "Neteisinga reiksme: ";
00095                 }
00096
00097                 std::cout << "Namu darbu skaicius: ";
00098                 while ( !skaitytiSveika( n, 1, 100 ) ) {
00099                     std::cout << "Neteisinga reiksme: ";
00100                 }
00101
00102                 std::vector<Studentas> studentai( m );
00103
00104                 for ( int i = 0; i < m; i++ )
00105                 {
00106                     generuotiVarda( studentai[i], i );
00107                     generuotiPazymius( studentai[i], n );
00108                 }
00109
00110                 spausdintiRezultatus( studentai, m, mediana );
00111                 break;
00112             }
00113
00114             case 4:
00115             {
00116                 std::vector<Studentas> studentai = nuskaitytiStudentus( );
00117
00118                 if ( studentai.size( ) > 0 )
00119                     spausdintiRezultatus( studentai, static_cast<int>( studentai.size( ) ), mediana );
00120                 else
00121                     std::cout << "Nera studentu duomenu.\n";
00122
00123                 break;

```

```

00124         }
00125
00126         case 5:
00127         {
00128             trymas1_failuKurimas( );
00129             break;
00130         }
00131
00132         case 6:
00133         {
00134             trymasKonteineriu( mediana );
00135             break;
00136         }
00137
00138         case 7:
00139         {
00140             trymasStrategiju( mediana );
00141             break;
00142         }
00143
00144         case 8:
00145         {
00146             testuotiKlase();
00147             break;
00148         }
00149
00150         case 9:
00151         {
00152             std::cout << "Programa baigta.\n";
00153             break;
00154         }
00155     }
00156 }
00157 catch ( const FailoKlaida& e )
00158 {
00159     std::cerr << "Failo klaida: " << e.what( ) << "\n";
00160 }
00161 catch ( const DuomenuKlaida& e )
00162 {
00163     std::cerr << "Duomenu klaida: " << e.what( ) << "\n";
00164 }
00165 catch ( const std::exception& e )
00166 {
00167     std::cerr << "Klaida: " << e.what( ) << "\n";
00168 }
00169
00170 } while ( meniu != 9 );
00171
00172 return 0;
00173 }

```

7.9 ConsoleApplication1/skaiciavimas.cpp File Reference

```
#include "skaiciavimas.h"
```

```
#include <algorithm>
```

Include dependency graph for skaiciavimas.cpp:

Functions

- double [skaiciuotiVidurki](#) (const std::vector< int > &nd, int n)
- double [skaiciuotiMediana](#) (const std::vector< int > &nd, int n)
- double [skaiciuotiGalutini](#) (double nd_rezultatas, int egz)

7.9.1 Function Documentation

7.9.1.1 skaiciuotiGalutini()

```
double skaiciuotiGalutini (
    double nd_rezultatas,
    int egz)
```

Definition at line 34 of file [skaiciavimas.cpp](#).

7.9.1.2 skaiciuotiMediana()

```
double skaiciuotiMediana (
    const std::vector< int > & nd,
    int n)
```

Definition at line 16 of file [skaiciavimas.cpp](#).

7.9.1.3 skaiciuotiVidurki()

```
double skaiciuotiVidurki (
    const std::vector< int > & nd,
    int n)
```

Definition at line 4 of file [skaiciavimas.cpp](#).

7.10 skaiciavimas.cpp

[Go to the documentation of this file.](#)

```
00001 #include "skaiciavimas.h"
00002 #include <algorithm>
00003
00004 double skaiciuotiVidurki( const std::vector<int>& nd, int n )
00005 {
00006     if ( n == 0 )
00007         return 0.0;
00008
00009     double suma = 0;
00010     for ( int i = 0; i < n; i++ )
00011         suma += nd [ i ];
00012
00013     return suma / n;
00014 }
00015
00016 double skaiciuotiMediana( const std::vector<int>& nd, int n )
00017 {
00018     if ( n == 0 )
00019         return 0.0;
00020
00021     std::vector<int> copy = nd;
00022     std::sort( copy.begin(), copy.end() );
00023
00024     double rezultatas;
00025
00026     if ( n % 2 == 0 )
00027         rezultatas = ( copy [ n / 2 - 1 ] + copy [ n / 2 ] ) / 2.0;
00028     else
00029         rezultatas = copy [ n / 2 ];
00030
00031     return rezultatas;
00032 }
00033
00034 double skaiciuotiGalutini( double nd_rezultatas, int egz )
00035 {
00036     return 0.4 * nd_rezultatas + 0.6 * egz;
00037 }
```

7.11 ConsoleApplication1/skaiciavimas.h File Reference

```
#include <vector>
```

Include dependency graph for skaiciavimas.h: This graph shows which files directly or indirectly include this file:

Functions

- double [skaiciuotiVidurki](#) (const std::vector< int > &nd, int n)
- double [skaiciuotiMediana](#) (const std::vector< int > &nd, int n)
- double [skaiciuotiGalutini](#) (double nd_rezultatas, int egz)

7.11.1 Function Documentation

7.11.1.1 skaiciuotiGalutini()

```
double skaiciuotiGalutini (
    double nd_rezultatas,
    int egz)
```

Definition at line 34 of file [skaiciavimas.cpp](#).

7.11.1.2 skaiciuotiMediana()

```
double skaiciuotiMediana (
    const std::vector< int > & nd,
    int n)
```

Definition at line 16 of file [skaiciavimas.cpp](#).

7.11.1.3 skaiciuotiVidurki()

```
double skaiciuotiVidurki (
    const std::vector< int > & nd,
    int n)
```

Definition at line 4 of file [skaiciavimas.cpp](#).

7.12 skaiciavimas.h

[Go to the documentation of this file.](#)

```
00001 #ifndef SKAICIAVIMAS_H
00002 #define SKAICIAVIMAS_H
00003
00004 #include <vector>
00005
00006 double skaiciuotiVidurki( const std::vector<int>& nd, int n );
00007 double skaiciuotiMediana( const std::vector<int>& nd, int n );
00008 double skaiciuotiGalutini( double nd_rezultatas, int egz );
00009
00010 #endif
```

7.13 ConsoleApplication1/studentas.cpp File Reference

```
#include "studentas.h"
#include <utility>
```

Include dependency graph for studentas.cpp:

Functions

- `std::ostream & operator<<` (`std::ostream &out`, `const Studentas &s`)
- `std::istream & operator>>` (`std::istream &in`, `Studentas &s`)

7.13.1 Function Documentation

7.13.1.1 `operator<<()`

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & s)
```

Definition at line 66 of file `studentas.cpp`.

7.13.1.2 `operator>>()`

```
std::istream & operator>> (
    std::istream & in,
    Studentas & s)
```

Definition at line 78 of file `studentas.cpp`.

7.14 `studentas.cpp`

[Go to the documentation of this file.](#)

```
00001 #include "studentas.h"
00002 #include <utility>
00003
00004 Studentas::Studentas()
00005     : Zmogus(), nd(), n( 0 ), egzaminas( 0 )
00006 {
00007 }
00008
00009 Studentas::~~Studentas() noexcept
00010 {
00011     n = 0;
00012     egzaminas = 0;
00013 }
00014
00015 Studentas::Studentas( const Studentas& kitas )
00016     : Zmogus( kitas ),
00017       nd( kitas.nd ),
00018       n( kitas.n ),
00019       egzaminas( kitas.egzaminas )
00020 {
00021 }
00022
00023 Studentas& Studentas::operator=( const Studentas& kitas )
00024 {
00025     if ( this == &kitas )
00026         return *this;
00027
00028     vardas = kitas.vardas;
00029     pavarde = kitas.pavarde;
00030     nd = kitas.nd;
00031     n = kitas.n;
00032     egzaminas = kitas.egzaminas;
00033
00034     return *this;
00035 }
00036
00037 Studentas::Studentas( Studentas&& kitas ) noexcept
```

```

00038     : Zmogus(),
00039     nd( std::move( kitas.nd ) ),
00040     n( kitas.n ),
00041     egzaminas( kitas.egzaminas )
00042 {
00043     vardas      = std::move( kitas.vardas );
00044     pavarde     = std::move( kitas.pavarde );
00045     kitas.n      = 0;
00046     kitas.egzaminas = 0;
00047 }
00048
00049 Studentas& Studentas::operator=( Studentas&& kitas ) noexcept
00050 {
00051     if ( this == &kitas )
00052         return *this;
00053
00054     vardas      = std::move( kitas.vardas );
00055     pavarde     = std::move( kitas.pavarde );
00056     nd          = std::move( kitas.nd );
00057     n           = kitas.n;
00058     egzaminas   = kitas.egzaminas;
00059
00060     kitas.n      = 0;
00061     kitas.egzaminas = 0;
00062
00063     return *this;
00064 }
00065
00066 std::ostream& operator<<( std::ostream& out, const Studentas& s )
00067 {
00068     out << s.vardas << " " << s.pavarde << " " << s.n;
00069
00070     for ( int i = 0; i < s.n; i++ )
00071         out << " " << s.nd[ i ];
00072
00073     out << " " << s.egzaminas;
00074
00075     return out;
00076 }
00077
00078 std::istream& operator>>( std::istream& in, Studentas& s )
00079 {
00080     if ( !( in >> s.vardas >> s.pavarde >> s.n ) )
00081         return in;
00082
00083     s.nd.resize( s.n );
00084
00085     for ( int i = 0; i < s.n; i++ )
00086     {
00087         if ( !( in >> s.nd[ i ] ) )
00088             return in;
00089     }
00090
00091     in >> s.egzaminas;
00092
00093     return in;
00094 }

```

7.15 ConsoleApplication1/studentas.h File Reference

```

#include "zmogus.h"
#include <vector>
#include <iostream>

```

Include dependency graph for studentas.h: This graph shows which files directly or indirectly include this file:

Data Structures

- class [Studentas](#)
Studento duomenų klase, paveldinti is [Zmogus](#).

Functions

- std::ostream & [operator<<](#) (std::ostream &out, const [Studentas](#) &s)
- std::istream & [operator>>](#) (std::istream &in, [Studentas](#) &s)

7.15.1 Function Documentation

7.15.1.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & s)
```

Definition at line 66 of file [studentas.cpp](#).

7.15.1.2 operator>>()

```
std::istream & operator>> (
    std::istream & in,
    Studentas & s)
```

Definition at line 78 of file [studentas.cpp](#).

7.16 studentas.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "zmogus.h"
00005 #include <vector>
00006 #include <iostream>
00007
00014 class Studentas : public Zmogus
00015 {
00016 public:
00017     std::vector<int> nd;
00018     int n;
00019     int egzaminas;
00020
00022     Studentas();
00023     ~Studentas() noexcept override;
00024
00026     Studentas( const Studentas& kitas );
00027     Studentas& operator=( const Studentas& kitas );
00028
00030     Studentas( Studentas&& kitas ) noexcept;
00031     Studentas& operator=( Studentas&& kitas ) noexcept;
00032 };
00033
00034 std::ostream& operator<<( std::ostream& out, const Studentas& s );
00035 std::istream& operator>>( std::istream& in, Studentas& s );
00036
00037 #endif
```

7.17 ConsoleApplication1/studentas_utils.cpp File Reference

```
#include "studentas_utils.h"
#include "skaiciavimas.h"
#include "io.h"
#include <iostream>
#include <algorithm>
#include <cstdlib>
```

Include dependency graph for [studentas_utils.cpp](#):

Functions

- void [generuotiPazymius](#) ([Studentas](#) &s, int n)
- void [generuotiVarda](#) ([Studentas](#) &s, int indeksas)
- int [pasirinktiRusiavima](#) ()
- void [rusiuotiStudentus](#) (std::vector< [Studentas](#) > &studentai, int rusiavimas)

7.17.1 Function Documentation

7.17.1.1 [generuotiPazymius\(\)](#)

```
void generuotiPazymius (  
    Studentas & s,  
    int n)
```

Definition at line 8 of file [studentas_utils.cpp](#).

7.17.1.2 [generuotiVarda\(\)](#)

```
void generuotiVarda (  
    Studentas & s,  
    int indeksas)
```

Definition at line 19 of file [studentas_utils.cpp](#).

7.17.1.3 [pasirinktiRusiavima\(\)](#)

```
int pasirinktiRusiavima ()
```

Definition at line 29 of file [studentas_utils.cpp](#).

7.17.1.4 [rusiuotiStudentus\(\)](#)

```
void rusiuotiStudentus (  
    std::vector< Studentas > & studentai,  
    int rusiavimas)
```

Definition at line 48 of file [studentas_utils.cpp](#).

7.18 studentas_utils.cpp

[Go to the documentation of this file.](#)

```

00001 #include "studentas_utils.h"
00002 #include "skaiciavimas.h"
00003 #include "io.h"
00004 #include <iostream>
00005 #include <algorithm>
00006 #include <cstdlib>
00007
00008 void generuotiPazymius( Studentas& s, int n )
00009 {
00010     s.n = n;
00011     s.nd.resize( n );
00012
00013     for ( int i = 0; i < n; i++ )
00014         s.nd [ i ] = rand( ) % 10 + 1;
00015
00016     s.egzaminas = rand( ) % 10 + 1;
00017 }
00018
00019 void generuotiVarda( Studentas& s, int indeksas )
00020 {
00021     std::string vardai[ ] = { "Jonas", "Petras", "Ona", "Marta", "Lukas", "Egle", "Tomas", "Inga" };
00022     std::string pavardes[ ] = { "Jonaitis", "Petraitis", "Kazlauskas", "Stankevicius", "Vaitkus",
    "Lukosius" };
00023
00024     s.vardas = vardai [ indeksas % 8 ];
00025     s.pavarde = pavardes [ indeksas % 6 ];
00026 }
00027
00028
00029 int pasirinktiRusiavima( )
00030 {
00031     int pasirinkimas;
00032
00033
00034     std::cout << "\nRusiavimo pasirinkimas:\n";
00035     std::cout << " 1 - Pagal vardą\n";
00036     std::cout << " 2 - Pagal pavardę\n";
00037     std::cout << " 3 - Pagal galutini (vidurkis)\n";
00038     std::cout << " 4 - Pagal galutini (mediana)\n";
00039     std::cout << "Pasirinkimas: ";
00040
00041     while ( !skaitytiSveika( pasirinkimas, 1, 4 ) ) {
00042         std::cout << "Neteisinga reikšmė. Pasirinkite 1-4: ";
00043     }
00044
00045     return pasirinkimas;
00046 }
00047
00048 void rusiuotiStudentus( std::vector<Studentas>& studentai, int rusiavimas )
00049 {
00050     switch ( rusiavimas ) {
00051     case 1:
00052         std::sort( studentai.begin( ), studentai.end( ), [ ] ( const Studentas& a, const Studentas& b
    ) {
00053             return a.vardas < b.vardas;
00054         } );
00055         break;
00056     case 2:
00057         std::sort( studentai.begin( ), studentai.end( ), [ ] ( const Studentas& a, const Studentas& b
    ) {
00058             return a.pavarde < b.pavarde;
00059         } );
00060         break;
00061     case 3:
00062         std::sort( studentai.begin( ), studentai.end( ), [ ] ( const Studentas& a, const Studentas& b
    ) {
00063             double ga = skaiciuotiGalutini( skaiciuotiVidurki( a.nd, a.n ), a.egzaminas );
00064             double gb = skaiciuotiGalutini( skaiciuotiVidurki( b.nd, b.n ), b.egzaminas );
00065             return ga > gb;
00066         } );
00067         break;
00068     case 4:
00069         std::sort( studentai.begin( ), studentai.end( ), [ ] ( const Studentas& a, const Studentas& b
    ) {
00070             double ga = skaiciuotiGalutini( skaiciuotiMediana( a.nd, a.n ), a.egzaminas );
00071             double gb = skaiciuotiGalutini( skaiciuotiMediana( b.nd, b.n ), b.egzaminas );
00072             return ga > gb;
00073         } );
00074         break;
00075     }
00076 }

```

7.19 ConsoleApplication1/studentas_utils.h File Reference

```
#include "studentas.h"
```

```
#include <vector>
```

Include dependency graph for studentas_utils.h: This graph shows which files directly or indirectly include this file:

Functions

- void [generuotiPazymius](#) ([Studentas](#) &s, int n)
- void [generuotiVarda](#) ([Studentas](#) &s, int indeksas)
- int [pasirinktiRusiavima](#) ()
- void [rusiuotiStudentus](#) (std::vector< [Studentas](#) > &studentai, int rusiavimas)

7.19.1 Function Documentation

7.19.1.1 [generuotiPazymius\(\)](#)

```
void generuotiPazymius (  
    Studentas & s,  
    int n)
```

Definition at line 8 of file [studentas_utils.cpp](#).

7.19.1.2 [generuotiVarda\(\)](#)

```
void generuotiVarda (  
    Studentas & s,  
    int indeksas)
```

Definition at line 19 of file [studentas_utils.cpp](#).

7.19.1.3 [pasirinktiRusiavima\(\)](#)

```
int pasirinktiRusiavima ()
```

Definition at line 29 of file [studentas_utils.cpp](#).

7.19.1.4 [rusiuotiStudentus\(\)](#)

```
void rusiuotiStudentus (  
    std::vector< Studentas > & studentai,  
    int rusiavimas)
```

Definition at line 48 of file [studentas_utils.cpp](#).

7.20 studentas_utils.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENTAS_UTILS_H
00002 #define STUDENTAS_UTILS_H
00003
00004 #include "studentas.h"
00005 #include <vector>
00006
00007 void generuotiPazymius( Studentas& s, int n );
00008 void generuotiVarda( Studentas& s, int indeksas );
00009 int pasirinktiRusiavima( );
00010 void rusiuotiStudentus( std::vector<Studentas>& studentai, int rusiavimas );
00011
00012
00013 #endif
```

7.21 ConsoleApplication1/testavimas.cpp File Reference

```
#include "testavimas.h"
#include "skaiciavimas.h"
#include "studentas_utils.h"
#include "exceptions.h"
#include "io.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <iomanip>
#include <chrono>
#include <cstdlib>
#include <string>
#include <algorithm>
#include <vector>
#include <list>
#include <deque>
Include dependency graph for testavimas.cpp:
```

Functions

- void [testuotiKlase](#) ()
- void [generuotiFaila](#) (const std::string &failoVardas, int irasu_sk, int nd_kiekis)
- std::vector< [Studentas](#) > [nuskaitytiFailo](#) (const std::string &failoVardas)
- void [isvestiKategorijaFaila](#) (const std::string &failoVardas, const std::vector< [Studentas](#) > &studentai, bool mediana)
- void [tyrimas1_failuKurimas](#) ()
- void [tyrimasKontaineriu](#) (bool mediana)
- void [tyrimasStrategiju](#) (bool mediana)

7.21.1 Function Documentation

7.21.1.1 generuotiFaila()

```
void generuotiFaila (
    const std::string & failoVardas,
    int irasu_sk,
    int nd_kiekis)
```

Definition at line [182](#) of file [testavimas.cpp](#).

7.21.1.2 isvestiKategorijaIFaila()

```
void isvestiKategorijaIFaila (  
    const std::string & failoVardas,  
    const std::vector< Studentas > & studentai,  
    bool mediana)
```

Definition at line 226 of file [testavimas.cpp](#).

7.21.1.3 nuskaitytiIsFailo()

```
std::vector< Studentas > nuskaitytiIsFailo (  
    const std::string & failoVardas)
```

Definition at line 221 of file [testavimas.cpp](#).

7.21.1.4 testuotiKlase()

```
void testuotiKlase ()
```

Definition at line 39 of file [testavimas.cpp](#).

7.21.1.5 tyrimas1_failuKurimas()

```
void tyrimas1_failuKurimas ()
```

Definition at line 262 of file [testavimas.cpp](#).

7.21.1.6 tyrimasKonteineriu()

```
void tyrimasKonteineriu (  
    bool mediana)
```

Definition at line 313 of file [testavimas.cpp](#).

7.21.1.7 tyrimasStrategiju()

```
void tyrimasStrategiju (  
    bool mediana)
```

Definition at line 385 of file [testavimas.cpp](#).

7.22 testavimas.cpp

[Go to the documentation of this file.](#)

```

00001 #include "testavimas.h"
00002 #include "skaiciavimas.h"
00003 #include "studentas_utils.h"
00004 #include "exceptions.h"
00005 #include "io.h"
00006
00007 #include <iostream>
00008 #include <fstream>
00009 #include <sstream>
00010 #include <iomanip>
00011 #include <chrono>
00012 #include <cstdlib>
00013 #include <string>
00014 #include <algorithm>
00015 #include <vector>
00016 #include <list>
00017 #include <deque>
00018
00019 static int g_ok = 0;
00020 static int g_fail = 0;
00021
00022 static void spausdintiRezultata( const std::string& testas, bool rezultatas )
00023 {
00024     std::cout << " " << ( rezultatas ? "[OK]" : "[FAIL]" ) << " " << testas << "\n";
00025     if ( rezultatas ) ++g_ok; else ++g_fail;
00026 }
00027
00028 static Studentas sukurtiTestoStudenta()
00029 {
00030     Studentas s;
00031     s.vardas = "Jonas";
00032     s.pavarde = "Jonaitis";
00033     s.n = 3;
00034     s.nd = { 8, 9, 7 };
00035     s.egzaminas = 10;
00036     return s;
00037 }
00038
00039 void testuotiKlase()
00040 {
00041     std::cout << "\n===== Studentas klases testai =====\n\n";
00042
00043     std::cout << "0. Zmogus abstrakti klase:\n";
00044     spausdintiRezultata( "std::is_abstract<Zmogus>::value == true",
00045                         std::is_abstract<Zmogus>::value == true );
00046     spausdintiRezultata( "std::is_abstract<Studentas>::value == false",
00047                         std::is_abstract<Studentas>::value == false );
00048     spausdintiRezultata( "Studentas yra isvestine is Zmogus",
00049                         std::is_base_of<Zmogus, Studentas>::value == true );
00050
00051     std::cout << "\n1. Default konstruktorius:\n";
00052     {
00053         Studentas s;
00054         spausdintiRezultata( "vardas == \"\", s.vardas == \"\" );
00055         spausdintiRezultata( "pavarde == \"\", s.pavarde == \"\" );
00056         spausdintiRezultata( "n == 0", s.n == 0 );
00057         spausdintiRezultata( "egzaminas == 0", s.egzaminas == 0 );
00058         spausdintiRezultata( "nd.empty()", s.nd.empty() );
00059     }
00060
00061     std::cout << "\n2. Kopijavimo konstruktorius:\n";
00062     {
00063         Studentas s1 = sukurtiTestoStudenta();
00064         Studentas s2( s1 );
00065
00066         spausdintiRezultata( "s2.vardas == \"Jonas\"", s2.vardas == "Jonas" );
00067         spausdintiRezultata( "s2.pavarde == \"Jonaitis\"", s2.pavarde == "Jonaitis" );
00068         spausdintiRezultata( "s2.n == 3", s2.n == 3 );
00069         spausdintiRezultata( "s2.egzaminas == 10", s2.egzaminas == 10 );
00070         spausdintiRezultata( "s2.nd == {8,9,7}", s2.nd == std::vector<int>{ 8, 9, 7 } );
00071     };
00072
00073     s2.vardas = "Petras";
00074     s2.nd[ 0 ] = 1;
00075     spausdintiRezultata( "originalas nepakito (vardas)", s1.vardas == "Jonas" );
00076     spausdintiRezultata( "originalas nepakito (nd[0])", s1.nd[ 0 ] == 8 );
00077 }
00078
00079 std::cout << "\n3. Kopijavimo priskyrimas (operator=):\n";
00080 {
00081     Studentas s1 = sukurtiTestoStudenta();
00082     Studentas s2;

```

```

00082         s2 = s1;
00083
00084         spausdintiRezultata( "s2.vardas == \"Jonas\"",      s2.vardas == "Jonas" );
00085         spausdintiRezultata( "s2.n == 3",                  s2.n == 3 );
00086         spausdintiRezultata( "s2.egzaminas == 10",          s2.egzaminas == 10 );
00087         spausdintiRezultata( "s2.nd == {8,9,7}",            s2.nd == std::vector<int>{ 8, 9, 7 }
);
00088
00089         s2.pavarde = "Kazlauskas";
00090         spausdintiRezultata( "originalas nepakito (pavarde)", s1.pavarde == "Jonaitis" );
00091
00092         s1 = s1;
00093         spausdintiRezultata( "savipriskyrimas saugus",      s1.vardas == "Jonas" );
00094     }
00095
00096     std::cout << "\n4. Perkélimo konstruktorius (move ctor):\n";
00097     {
00098         Studentas s1 = sukurtiTestoStudenta();
00099         Studentas s2( std::move( s1 ) );
00100
00101         spausdintiRezultata( "s2.vardas == \"Jonas\"",      s2.vardas == "Jonas" );
00102         spausdintiRezultata( "s2.n == 3",                  s2.n == 3 );
00103         spausdintiRezultata( "s2.egzaminas == 10",          s2.egzaminas == 10 );
00104         spausdintiRezultata( "s2.nd == {8,9,7}",            s2.nd == std::vector<int>{ 8, 9, 7 }
);
00105
00106         spausdintiRezultata( "s1.n == 0 (saltinis istusejo)", s1.n == 0 );
00107         spausdintiRezultata( "s1.egzaminas == 0",            s1.egzaminas == 0 );
00108     }
00109     std::cout << "\n5. Perkélimo priskyrimas (move operator):\n";
00110     {
00111         Studentas s1 = sukurtiTestoStudenta();
00112         Studentas s2;
00113         s2 = std::move( s1 );
00114
00115         spausdintiRezultata( "s2.vardas == \"Jonas\"",      s2.vardas == "Jonas" );
00116         spausdintiRezultata( "s2.n == 3",                  s2.n == 3 );
00117         spausdintiRezultata( "s2.egzaminas == 10",          s2.egzaminas == 10 );
00118         spausdintiRezultata( "s2.nd == {8,9,7}",            s2.nd == std::vector<int>{ 8, 9, 7 }
);
00119
00120         spausdintiRezultata( "s1.n == 0 (saltinis istusejo)", s1.n == 0 );
00121         spausdintiRezultata( "s1.egzaminas == 0",            s1.egzaminas == 0 );
00122
00123         s2 = std::move( s2 );
00124         spausdintiRezultata( "saviperkélimas saugus", s2.n == 3 );
00125     }
00126     std::cout << "\n6. operator« (isvedimas):\n";
00127     {
00128         Studentas s = sukurtiTestoStudenta();
00129         std::ostringstream oss;
00130         oss << s;
00131         std::string rezultatas = oss.str();
00132         std::string laukiamas = "Jonas Jonaitis 3 8 9 7 10";
00133         spausdintiRezultata( "operator« ekrane: \"\" + rezultatas + \"\"",
                                rezultatas == laukiamas );
00134
00135         std::ofstream failas( "test_studentas.txt" );
00136         failas << s;
00137         failas.close();
00138         spausdintiRezultata( "operator« irasė i faila", true );
00139     }
00140
00141     std::cout << "\n7. operator» (ivedimas):\n";
00142     {
00143         std::istringstream iss( "Ona Kazlauskiene 2 6 8 9" );
00144         Studentas s;
00145         iss >> s;
00146         spausdintiRezultata( "vardas == \"Ona\"", s.vardas == "Ona" );
00147         spausdintiRezultata( "pavarde == \"Kazlauskiene\"", s.pavarde == "Kazlauskiene" );
00148         spausdintiRezultata( "n == 2", s.n == 2 );
00149         spausdintiRezultata( "nd == {6,8}", s.nd == std::vector<int>{ 6, 8 } );
00150         spausdintiRezultata( "egzaminas == 9", s.egzaminas == 9 );
00151
00152         std::ifstream failas( "test_studentas.txt" );
00153         Studentas sIsFailo;
00154         failas >> sIsFailo;
00155         failas.close();
00156         spausdintiRezultata( "operator» is failo: vardas == \"Jonas\"",
                                sIsFailo.vardas == "Jonas" );
00157         spausdintiRezultata( "operator» is failo: n == 3",
                                sIsFailo.n == 3 );
00158     }
00159     std::cout << "\n8. Destruktorius:\n";
00160     {
00161     }
00162
00163     std::cout << "\n8. Destruktorius:\n";
00164     {
00165     }

```

```

00166         Studentas s = sukurtiTestoStudenta();
00167     }
00168     spausdintiRezultata( "destruktorius iskvietas be klaidu (scope pabaiga)", true );
00169 }
00170
00171     std::cout << "\n=====\\n";
00172     std::cout << "Rezultatai: " << g_ok << " OK, " << g_fail << " FAIL\\n";
00173     if ( g_fail == 0 )
00174         std::cout << "Visi testai praejo sekmingai!\\n\\n";
00175     else
00176         std::cout << "Kai kurie testai nepraejo!\\n\\n";
00177
00178     g_ok = 0;
00179     g_fail = 0;
00180 }
00181
00182 void generuotiFaila( const std::string& failoVardas, int irasu_sk, int nd_kiekis )
00183 {
00184     std::ofstream out( failoVardas );
00185     if ( !out.is_open( ) )
00186         throw FailoKlaida( "Nepavyko sukurti failo: " + failoVardas );
00187
00188     out << std::left
00189         << std::setw( 25 ) << "Vardas"
00190         << std::setw( 27 ) << "Pavarde";
00191
00192     std::stringstream a2;
00193
00194     for ( int i = 1; i <= nd_kiekis; i++ )
00195         a2 << std::setw( 10 ) << ( "ND" + std::to_string( i ) );
00196
00197     out << a2.str( );
00198
00199     out << std::setw( 10 ) << "Egz." << "\\n";
00200
00201     for ( int i = 1; i <= irasu_sk; i++ )
00202     {
00203         std::stringstream a3;
00204
00205         a3 << std::left
00206             << std::setw( 25 ) << ( "Vardas" + std::to_string( i ) )
00207             << std::setw( 27 ) << ( "Pavarde" + std::to_string( i ) )
00208             << std::right;
00209
00210         for ( int j = 0; j < nd_kiekis; j++ )
00211             a3 << std::setw( 10 ) << ( rand( ) % 10 + 1 );
00212
00213         a3 << std::setw( 10 ) << ( rand( ) % 10 + 1 ) << "\\n";
00214
00215         out << a3.str( );
00216     }
00217
00218     out.close( );
00219 }
00220
00221 std::vector<Studentas> nuskaitytiIsFailo( const std::string& failoVardas )
00222 {
00223     return nuskaitytiIsFailoT<std::vector<Studentas>>( failoVardas );
00224 }
00225
00226 void isvestiKategorijaIFaila( const std::string& failoVardas,
00227     const std::vector<Studentas>& studentai, bool mediana )
00228 {
00229     std::ofstream out( failoVardas );
00230     if ( !out.is_open( ) )
00231         throw FailoKlaida( "Nepavyko sukurti failo: " + failoVardas );
00232
00233     out << std::left
00234         << std::setw( 20 ) << "Pavarde"
00235         << std::setw( 20 ) << "Vardas"
00236         << std::setw( 20 ) << "Galutinis" << "\\n";
00237     out << std::string( 60, '-' ) << "\\n";
00238
00239     for ( const auto& s : studentai )
00240     {
00241         double galutinis;
00242
00243         if ( mediana )
00244             galutinis = skaiciuotiGalutini( skaiciuotiMediana( s.nd, s.n ), s.egzaminas );
00245         else
00246             galutinis = skaiciuotiGalutini( skaiciuotiVidurki( s.nd, s.n ), s.egzaminas );
00247
00248         std::stringstream ss;
00249
00250         ss << std::left
00251             << std::setw( 20 ) << s.pavarde
00252             << std::setw( 20 ) << s.vardas

```

```

00253         « std::fixed « std::setprecision( 2 )
00254         « std::setw( 20 ) « galutinis « "\n";
00255
00256     out « ss.str( );
00257 }
00258
00259 out.close( );
00260 }
00261
00262 void tyrimasI_failuKurimas( )
00263 {
00264     const int dydziai[] = { 1000, 10000, 100000, 1000000, 10000000 };
00265     const std::string pavadinimai[] = {
00266         "studentai_1000.txt",
00267         "studentai_10000.txt",
00268         "studentai_100000.txt",
00269         "studentai_1000000.txt",
00270         "studentai_10000000.txt"
00271     };
00272     const int nd_kiekis = 15;
00273     const int bandymu_sk = 3;
00274
00275     std::cout « "\n===== 1 TYRIMAS: Failu kurimas =====\n\n";
00276
00277     std::cout « std::left
00278         « std::setw( 18 ) « "Irasu sk."
00279         « std::setw( 18 ) « "1 bandymas (s)"
00280         « std::setw( 18 ) « "2 bandymas (s)"
00281         « std::setw( 18 ) « "3 bandymas (s)"
00282         « std::setw( 18 ) « "Vidurkis (s)" « "\n";
00283     std::cout « std::string( 90, '-' ) « "\n";
00284
00285     for ( int i = 0; i < 5; i++ )
00286     {
00287         double laikai[bandymu_sk] = { };
00288         double suma = 0.0;
00289
00290         for ( int b = 0; b < bandymu_sk; b++ )
00291         {
00292             std::remove( pavadinimai[i].c_str( ) );
00293
00294             auto start = std::chrono::high_resolution_clock::now( );
00295             generuotiFaila( pavadinimai[i], dydziai[i], nd_kiekis );
00296             auto end = std::chrono::high_resolution_clock::now( );
00297
00298             laikai[b] = std::chrono::duration<double>( end - start ).count( );
00299             suma += laikai[b];
00300         }
00301
00302         std::cout « std::left « std::setw( 18 ) « dydziai[i]
00303             « std::fixed « std::setprecision( 5 )
00304             « std::setw( 18 ) « laikai[0]
00305             « std::setw( 18 ) « laikai[1]
00306             « std::setw( 18 ) « laikai[2]
00307             « std::setw( 18 ) « ( suma / bandymu_sk ) « "\n";
00308     }
00309
00310     std::cout « "\nFailai sugeneruoti ir issaugoti.\n";
00311 }
00312
00313 void tyrimasKonteineriu( bool mediana )
00314 {
00315     const int dydziai[] = { 1000, 10000, 100000, 1000000, 10000000 };
00316     const std::string pavadinimai[] = {
00317         "studentai_1000.txt",
00318         "studentai_10000.txt",
00319         "studentai_100000.txt",
00320         "studentai_1000000.txt",
00321         "studentai_10000000.txt"
00322     };
00323     const int bandymu_sk = 3;
00324
00325     std::cout « "\n===== 2 TYRIMAS: Konteineriu palyginimas =====\n";
00326     std::cout « "Kiekvienas matavimas atliktas " « bandymu_sk
00327         « " kartus, pateikiamas vidurkis.\n\n";
00328
00329     std::cout « std::left
00330         « std::setw( 12 ) « "Irasu sk."
00331         « std::setw( 14 ) « "Konteineris"
00332         « std::setw( 18 ) « "Nuskaitymas(s)"
00333         « std::setw( 18 ) « "Rusiavimas(s)"
00334         « std::setw( 18 ) « "Skaidymas(s)"
00335         « "\n";
00336     std::cout « std::string( 80, '-' ) « "\n";
00337
00338     for ( int i = 0; i < 5; i++ )
00339     {

```

```

00340     {
00341         std::ifstream test( pavadinimai[i] );
00342         if ( !test.is_open( ) )
00343         {
00344             std::cout << std::left << std::setw( 12 ) << dydziai[i]
00345                 << "Failas nerastas! Pirma paleiskite 1 tyrima.\n";
00346             continue;
00347         }
00348     }
00349
00350     double nusk, rus, skaid;
00351
00352     benchmarkKonteineris<std::vector<Studentas>>>(
00353         pavadinimai[i], mediana, bandymu_sk, nusk, rus, skaid );
00354     std::cout << std::left << std::setw( 12 ) << dydziai[i]
00355         << std::setw( 14 ) << "vector"
00356         << std::fixed << std::setprecision( 5 )
00357         << std::setw( 18 ) << nusk
00358         << std::setw( 18 ) << rus
00359         << std::setw( 18 ) << skaid << "\n";
00360
00361     benchmarkKonteineris<std::list<Studentas>>>(
00362         pavadinimai[i], mediana, bandymu_sk, nusk, rus, skaid );
00363     std::cout << std::left << std::setw( 12 ) << ""
00364         << std::setw( 14 ) << "list"
00365         << std::fixed << std::setprecision( 5 )
00366         << std::setw( 18 ) << nusk
00367         << std::setw( 18 ) << rus
00368         << std::setw( 18 ) << skaid << "\n";
00369
00370     benchmarkKonteineris<std::deque<Studentas>>>(
00371         pavadinimai[i], mediana, bandymu_sk, nusk, rus, skaid );
00372     std::cout << std::left << std::setw( 12 ) << ""
00373         << std::setw( 14 ) << "deque"
00374         << std::fixed << std::setprecision( 5 )
00375         << std::setw( 18 ) << nusk
00376         << std::setw( 18 ) << rus
00377         << std::setw( 18 ) << skaid << "\n";
00378
00379     std::cout << std::string( 80, '-' ) << "\n";
00380 }
00381
00382 std::cout << "\nTyrimas baigtas.\n";
00383 }
00384
00385 void tyrimasStrategiju( bool mediana )
00386 {
00387     const int dydziai[] = { 1000, 10000, 100000, 1000000, 10000000 };
00388     const std::string pavadinimai[] = {
00389         "studentai_1000.txt",
00390         "studentai_10000.txt",
00391         "studentai_100000.txt",
00392         "studentai_1000000.txt",
00393         //"studentai_10000000.txt"
00394     };
00395
00396     const int bandymu_sk = 1;
00397
00398     std::cout << "\n===== 3 TYRIMAS: Strategiju palyginimas =====\n";
00399     std::cout << "Kiekvienas matavimas atliktas " << bandymu_sk
00400         << " kartus, pateikiamas vidurkis.\n";
00401     std::cout << "Matuojamas TIK skaidymo i grupes laikas (be nuskaitymo ir rusiavimo).\n";
00402
00403     const char* konteineriai[] = { "std::vector", "std::list", "std::deque" };
00404
00405     for ( int k = 0; k < 3; k++ )
00406     {
00407         std::cout << "\n--- " << konteineriai[k] << " ---\n\n";
00408         std::cout << std::left
00409             << std::setw( 15 ) << "Irasu sk."
00410             << std::setw( 20 ) << "1 strategija(s)"
00411             << std::setw( 20 ) << "2 strategija(s)"
00412             << std::setw( 20 ) << "3 strategija(s)"
00413             << "\n";
00414         std::cout << std::string( 75, '-' ) << "\n";
00415
00416         for ( int i = 0; i < sizeof( pavadinimai ) / sizeof( pavadinimai[0]); i++ )
00417         {
00418             {
00419                 std::ifstream test( pavadinimai[i] );
00420                 if ( !test.is_open( ) )
00421                 {
00422                     std::cout << std::left << std::setw( 15 ) << dydziai[i]
00423                         << "Failas nerastas!\n";
00424                     continue;
00425                 }
00426             }

```

```

00427
00428         double s1, s2, s3;
00429
00430         if ( k == 0 )
00431             benchmarkStrategijos<std::vector<Studentas>>>(
00432                 pavadinimai[i], mediana, bandymu_sk, s1, s2, s3 );
00433         else if ( k == 1 )
00434             benchmarkStrategijos<std::list<Studentas>>>(
00435                 pavadinimai[i], mediana, bandymu_sk, s1, s2, s3 );
00436         else
00437             benchmarkStrategijos<std::deque<Studentas>>>(
00438                 pavadinimai[i], mediana, bandymu_sk, s1, s2, s3 );
00439
00440         std::cout << std::left << std::setw( 15 ) << dydziai[i]
00441             << std::fixed << std::setprecision( 5 )
00442             << std::setw( 20 ) << s1
00443             << std::setw( 20 ) << s2
00444             << std::setw( 20 ) << s3 << "\n";
00445     }
00446 }
00447
00448 std::cout << "\nTyrimas baigtas.\n";
00449
00450 std::cout << "\n--- Kietiakiai/vargsiukai issaugojimas i failus ---\n";
00451
00452 bool rastasVienas = false;
00453 for ( int i = 0; i < 5; i++ )
00454 {
00455     std::ifstream test( pavadinimai[i] );
00456     if ( !test.is_open( ) )
00457         continue;
00458     test.close( );
00459
00460     rastasVienas = true;
00461     std::string sk = std::to_string( dydziai[i] );
00462
00463     std::vector<Studentas> base = nuskaitytiIsFailoT<std::vector<Studentas>>>( pavadinimai[i] );
00464     rusiuotiPagalGalutini( base, mediana );
00465
00466     std::vector<Studentas> data = base;
00467     std::vector<Studentas> kiet, varg;
00468     strategijal( data, kiet, varg, mediana );
00469
00470     isvestiKategorijaIFaila( "kietiakiai_" + sk + ".txt", kiet, mediana );
00471     isvestiKategorijaIFaila( "vargsiukai_" + sk + ".txt", varg, mediana );
00472
00473     std::cout << " " << std::setw( 10 ) << std::left << sk << " irasu -> "
00474         << "kietiakiai_" << sk << ".txt (" << kiet.size( ) << " ), "
00475         << "vargsiukai_" << sk << ".txt (" << varg.size( ) << " )\n";
00476 }
00477
00478 if ( !rastasVienas )
00479     std::cout << " Nera failu - pirma paleiskite 1 tyrima.\n";
00480 }

```

7.23 ConsoleApplication1/testavimas.h File Reference

```

#include "studentas.h"
#include "zmogus.h"
#include "skaiciavimas.h"
#include "exceptions.h"
#include <string>
#include <vector>
#include <list>
#include <deque>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <type_traits>
#include <chrono>
#include <iostream>
#include <iomanip>
#include <iterator>

```

```
#include <typeinfo>
```

Include dependency graph for `testavimas.h`: This graph shows which files directly or indirectly include this file:

Functions

- void `generuotiFaila` (const std::string &failoVardas, int irasu_sk, int nd_kiekis)
- void `tyrimas1_failuKurimas` ()
- void `tyrimasKonteineriu` (bool mediana)
- void `tyrimasStrategiju` (bool mediana)
- void `testuotiKlase` ()
- std::vector< `Studentas` > `nuskaitytiFailo` (const std::string &failoVardas)
- void `isvestiKategorijaFaila` (const std::string &failoVardas, const std::vector< `Studentas` > &studentai, bool mediana)
- template<typename Container>
Container `nuskaitytiFailoT` (const std::string &failoVardas)
- double `apskaiciuotiGalutiniBala` (const `Studentas` &s, bool mediana)
- template<typename Container>
void `rusiuotiPagalGalutini` (Container &c, bool mediana)
- template<typename Container>
void `strategija1` (const Container &studentai, Container &kietiakiai, Container &vargsiukai, bool mediana)
- template<typename Container>
void `strategija2` (Container &studentai, Container &vargsiukai, bool mediana)
- template<typename Container>
void `strategija3` (Container &studentai, Container &vargsiukai, bool mediana)
- template<typename Container>
void `benchmarkKonteineris` (const std::string &failas, bool mediana, int bandymu_sk, double &nusk_avg, double &rus_avg, double &skaid_avg)
- template<typename Container>
void `benchmarkStrategijos` (const std::string &failas, bool mediana, int bandymu_sk, double &str1_avg, double &str2_avg, double &str3_avg)

7.23.1 Function Documentation

7.23.1.1 `apskaiciuotiGalutiniBala()`

```
double apskaiciuotiGalutiniBala (
    const Studentas & s,
    bool mediana) [inline]
```

Definition at line 87 of file `testavimas.h`.

7.23.1.2 `benchmarkKonteineris()`

```
template<typename Container>
void benchmarkKonteineris (
    const std::string & failas,
    bool mediana,
    int bandymu_sk,
    double & nusk_avg,
    double & rus_avg,
    double & skaid_avg)
```

Definition at line 153 of file `testavimas.h`.

7.23.1.3 benchmarkStrategijos()

```
template<typename Container>
void benchmarkStrategijos (
    const std::string & failas,
    bool mediana,
    int bandymu_sk,
    double & str1_avg,
    double & str2_avg,
    double & str3_avg)
```

Definition at line 182 of file [testavimas.h](#).

7.23.1.4 generuotiFaila()

```
void generuotiFaila (
    const std::string & failoVardas,
    int irasu_sk,
    int nd_kiekis)
```

Definition at line 182 of file [testavimas.cpp](#).

7.23.1.5 isvestiKategorijaIFaila()

```
void isvestiKategorijaIFaila (
    const std::string & failoVardas,
    const std::vector< Studentas > & studentai,
    bool mediana)
```

Definition at line 226 of file [testavimas.cpp](#).

7.23.1.6 nuskaitytiIsFailo()

```
std::vector< Studentas > nuskaitytiIsFailo (
    const std::string & failoVardas)
```

Definition at line 221 of file [testavimas.cpp](#).

7.23.1.7 nuskaitytiIsFailoT()

```
template<typename Container>
Container nuskaitytiIsFailoT (
    const std::string & failoVardas)
```

Definition at line 35 of file [testavimas.h](#).

7.23.1.8 rusiuotiPagalGalutini()

```
template<typename Container>
void rusiuotiPagalGalutini (
    Container & c,
    bool mediana)
```

Definition at line 96 of file [testavimas.h](#).

7.23.1.9 strategija1()

```
template<typename Container>
void strategija1 (
    const Container & studentai,
    Container & kietiakiai,
    Container & vargsiukai,
    bool mediana)
```

Definition at line 109 of file [testavimas.h](#).

7.23.1.10 strategija2()

```
template<typename Container>
void strategija2 (
    Container & studentai,
    Container & vargsiukai,
    bool mediana)
```

Definition at line 122 of file [testavimas.h](#).

7.23.1.11 strategija3()

```
template<typename Container>
void strategija3 (
    Container & studentai,
    Container & vargsiukai,
    bool mediana)
```

Definition at line 132 of file [testavimas.h](#).

7.23.1.12 testuotiKlase()

```
void testuotiKlase ()
```

Definition at line 39 of file [testavimas.cpp](#).

7.23.1.13 tyrimas1_failuKurimas()

```
void tyrimas1_failuKurimas ()
```

Definition at line 262 of file [testavimas.cpp](#).

7.23.1.14 tyrimasKonteineriu()

```
void tyrimasKonteineriu (
    bool mediana)
```

Definition at line 313 of file [testavimas.cpp](#).

7.23.1.15 tyrimasStrategiju()

```
void tyrimasStrategiju (
    bool mediana)
```

Definition at line 385 of file [testavimas.cpp](#).

7.24 testavimas.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TESTAVIMAS_H
00002 #define TESTAVIMAS_H
00003
00004 #include "studentas.h"
00005 #include "zmogus.h"
00006 #include "skaiciavimas.h"
00007 #include "exceptions.h"
00008
00009 #include <string>
00010 #include <vector>
00011 #include <list>
00012 #include <deque>
00013 #include <fstream>
00014 #include <sstream>
00015 #include <algorithm>
00016 #include <type_traits>
00017 #include <chrono>
00018 #include <iostream>
00019 #include <iomanip>
00020 #include <iterator>
00021 #include <typeinfo>
00022
00023 void generuotiFaila( const std::string& failoVardas, int irasu_sk, int nd_kiekis );
00024 void tyrimas1_failuKurimas( );
00025 void tyrimasKonteineriu( bool mediana );
00026 void tyrimasStrategiju( bool mediana );
00027 void testuotiKlase( );
00028
00029 std::vector<Studentas> nuskaitytiIsFailo( const std::string& failoVardas );
00030
00031 void isvestiKategorijaIFaila( const std::string& failoVardas,
00032     const std::vector<Studentas>& studentai, bool mediana );
00033
00034 template<typename Container>
00035 Container nuskaitytiIsFailoT( const std::string& failoVardas )
00036 {
00037     std::ifstream stream( failoVardas );
00038     if ( !stream.is_open( ) )
00039         throw FailoKlaida( "Nepavyko atidaryti failo: " + failoVardas );
00040
00041     Container out;
00042     std::string line;
00043
00044     std::stringstream a2;
00045     a2 << stream.rdbuf( );
00046
00047     if ( !std::getline( a2, line ) )
00048         return out;
00049
00050     std::stringstream header_stream( line );
00051     std::string column;
00052     std::vector<std::string> cols;
00053
00054     while ( header_stream >> column )
```

```

00055         cols.push_back( column );
00056
00057         size_t nd_count = cols.size() >= 3 ? cols.size() - 3 : 0;
00058
00059         while ( std::getline( a2, line ) )
00060         {
00061             if ( line.empty() )
00062                 continue;
00063
00064             std::stringstream ss( line );
00065             Studentas s;
00066             s.n = static_cast<int>( nd_count );
00067             s.nd.resize( nd_count );
00068
00069             if ( !( ss » s.vardas » s.pavarde ) )
00070                 throw DuomenuKlaida( "Nepavyko nuskaityti: " + line );
00071
00072             for ( size_t j = 0; j < nd_count; j++ )
00073             {
00074                 if ( !( ss » s.nd[j] ) )
00075                     throw DuomenuKlaida( "Nepavyko nuskaityti ND: " + line );
00076             }
00077
00078             if ( !( ss » s.egzaminas ) )
00079                 throw DuomenuKlaida( "Nepavyko nuskaityti egzamino: " + line );
00080
00081             out.push_back( s );
00082         }
00083
00084         return out;
00085     }
00086
00087 inline double apskaiciuotiGalutiniBala( const Studentas& s, bool mediana )
00088 {
00089     if ( mediana )
00090         return skaiciuotiGalutini( skaiciuotiMediana( s.nd, s.n ), s.egzaminas );
00091     else
00092         return skaiciuotiGalutini( skaiciuotiVidurki( s.nd, s.n ), s.egzaminas );
00093 }
00094
00095 template<typename Container>
00096 void rusiuotiPagalGalutini( Container& c, bool mediana )
00097 {
00098     auto comp = [mediana]( const Studentas& a, const Studentas& b ) {
00099         return apskaiciuotiGalutiniBala( a, mediana ) > apskaiciuotiGalutiniBala( b, mediana );
00100     };
00101
00102     if constexpr ( std::is_same_v<Container, std::list<Studentas>> )
00103         c.sort( comp );
00104     else
00105         std::sort( c.begin(), c.end(), comp );
00106 }
00107
00108 template<typename Container>
00109 void strategija1( const Container& studentai, Container& kietiakai,
00110                 Container& vargsiukai, bool mediana )
00111 {
00112     for ( const auto& s : studentai )
00113     {
00114         if ( apskaiciuotiGalutiniBala( s, mediana ) >= 5.0 )
00115             kietiakai.push_back( s );
00116         else
00117             vargsiukai.push_back( s );
00118     }
00119 }
00120
00121 template<typename Container>
00122 void strategija2( Container& studentai, Container& vargsiukai, bool mediana )
00123 {
00124     while ( apskaiciuotiGalutiniBala( studentai.back(), mediana ) < 5.0 )
00125     {
00126         vargsiukai.push_back( studentai.back() );
00127         studentai.pop_back();
00128     }
00129 }
00130
00131 template<typename Container>
00132 void strategija3( Container& studentai, Container& vargsiukai, bool mediana )
00133 {
00134     auto it = std::stable_partition( studentai.begin(), studentai.end(),
00135                                     [mediana]( const Studentas& s ) {
00136                                         return apskaiciuotiGalutiniBala( s, mediana ) >= 5.0;
00137                                     } );
00138
00139     if constexpr ( std::is_same_v<Container, std::list<Studentas>> )
00140     {
00141         vargsiukai.splice( vargsiukai.end(), studentai, it, studentai.end() );

```

```

00142     }
00143     else
00144     {
00145         vargsiukai.insert( vargsiukai.end( ),
00146             std::make_move_iterator( it ),
00147             std::make_move_iterator( studentai.end( ) ) );
00148         studentai.erase( it, studentai.end( ) );
00149     }
00150 }
00151
00152 template<typename Container>
00153 void benchmarkKonteineris( const std::string& failas, bool mediana, int bandymu_sk,
00154     double& nusk_avg, double& rus_avg, double& skaid_avg )
00155 {
00156     double nusk_suma = 0, rus_suma = 0, skaid_suma = 0;
00157
00158     for ( int b = 0; b < bandymu_sk; b++ )
00159     {
00160         auto t1 = std::chrono::high_resolution_clock::now( );
00161         Container data = nuskaitytiIsFailoT<Container>( failas );
00162         auto t2 = std::chrono::high_resolution_clock::now( );
00163
00164         rusiuotiPagalGalutini( data, mediana );
00165         auto t3 = std::chrono::high_resolution_clock::now( );
00166
00167         Container kiet, varg;
00168         strategija1( data, kiet, varg, mediana );
00169         auto t4 = std::chrono::high_resolution_clock::now( );
00170
00171         nusk_suma += std::chrono::duration<double>( t2 - t1 ).count( );
00172         rus_suma += std::chrono::duration<double>( t3 - t2 ).count( );
00173         skaid_suma += std::chrono::duration<double>( t4 - t3 ).count( );
00174     }
00175
00176     nusk_avg = nusk_suma / bandymu_sk;
00177     rus_avg = rus_suma / bandymu_sk;
00178     skaid_avg = skaid_suma / bandymu_sk;
00179 }
00180
00181 template<typename Container>
00182 void benchmarkStrategijos( const std::string& failas, bool mediana, int bandymu_sk,
00183     double& str1_avg, double& str2_avg, double& str3_avg )
00184 {
00185     Container original = nuskaitytiIsFailoT<Container>( failas );
00186     rusiuotiPagalGalutini( original, mediana );
00187
00188     double suma1 = 0, suma2 = 0, suma3 = 0;
00189
00190     for ( int b = 0; b < bandymu_sk; b++ )
00191     {
00192         {
00193             Container data = original;
00194             Container kiet, varg;
00195             auto t1 = std::chrono::high_resolution_clock::now( );
00196             strategija1( data, kiet, varg, mediana );
00197             auto t2 = std::chrono::high_resolution_clock::now( );
00198             suma1 += std::chrono::duration<double>( t2 - t1 ).count( );
00199         }
00200         {
00201             Container data = original;
00202             Container varg;
00203             auto t1 = std::chrono::high_resolution_clock::now( );
00204             strategija2( data, varg, mediana );
00205             auto t2 = std::chrono::high_resolution_clock::now( );
00206             suma2 += std::chrono::duration<double>( t2 - t1 ).count( );
00207         }
00208         {
00209             Container data = original;
00210             Container varg;
00211             auto t1 = std::chrono::high_resolution_clock::now( );
00212             strategija3( data, varg, mediana );
00213             auto t2 = std::chrono::high_resolution_clock::now( );
00214             suma3 += std::chrono::duration<double>( t2 - t1 ).count( );
00215         }
00216     }
00217
00218     str1_avg = suma1 / bandymu_sk;
00219     str2_avg = suma2 / bandymu_sk;
00220     str3_avg = suma3 / bandymu_sk;
00221 }
00222
00223 #endif

```

7.25 ConsoleApplication1/zmogus.cpp File Reference

```
#include "zmogus.h"
```

Include dependency graph for zmogus.cpp:

7.26 zmogus.cpp

[Go to the documentation of this file.](#)

```
00001 #include "zmogus.h"
00002
00003 Zmogus::Zmogus() : vardas( "" ), pavarde( "" )
00004 {
00005 }
00006
00007 Zmogus::~Zmogus()
00008 {
00009 }
```

7.27 ConsoleApplication1/zmogus.h File Reference

```
#include <string>
#include <iostream>
```

Include dependency graph for zmogus.h: This graph shows which files directly or indirectly include this file:

Data Structures

- class [Zmogus](#)

7.28 zmogus.h

[Go to the documentation of this file.](#)

```
00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005 #include <iostream>
00006
00007 class Zmogus
00008 {
00009 public:
00010     std::string vardas;
00011     std::string pavarde;
00012
00013     Zmogus();
00014     virtual ~Zmogus() = 0;
00015 };
00016
00017 #endif
```

Index

- ~Studentas
 - Studentas, [13](#)
- ~Zmogus
 - Zmogus, [15](#)
- apskaiciuotiGalutiniBala
 - testavimas.h, [42](#)
- benchmarkKonteineris
 - testavimas.h, [42](#)
- benchmarkStrategijos
 - testavimas.h, [42](#)
- ConsoleApplication1 Directory Reference, [9](#)
- ConsoleApplication1/exceptions.h, [17](#)
- ConsoleApplication1/io.cpp, [18](#), [19](#)
- ConsoleApplication1/io.h, [21](#), [22](#)
- ConsoleApplication1/main.cpp, [23](#)
- ConsoleApplication1/skaiciavimas.cpp, [25](#), [26](#)
- ConsoleApplication1/skaiciavimas.h, [26](#), [27](#)
- ConsoleApplication1/studentas.cpp, [27](#), [28](#)
- ConsoleApplication1/studentas.h, [29](#), [30](#)
- ConsoleApplication1/studentas_utils.cpp, [30](#), [32](#)
- ConsoleApplication1/studentas_utils.h, [33](#), [34](#)
- ConsoleApplication1/testavimas.cpp, [34](#), [36](#)
- ConsoleApplication1/testavimas.h, [41](#), [45](#)
- ConsoleApplication1/zmogus.cpp, [48](#)
- ConsoleApplication1/zmogus.h, [48](#)
- DuomenuKlaida, [11](#)
 - DuomenuKlaida, [11](#)
- egzaminas
 - Studentas, [14](#)
- FailoKlaida, [12](#)
 - FailoKlaida, [12](#)
- generuotiFaila
 - testavimas.cpp, [34](#)
 - testavimas.h, [43](#)
- generuotiPazymius
 - studentas_utils.cpp, [31](#)
 - studentas_utils.h, [33](#)
- generuotiVarda
 - studentas_utils.cpp, [31](#)
 - studentas_utils.h, [33](#)
- io.cpp
 - ivestiRankiniu, [18](#)
 - nuskaitytiStudentus, [18](#)
- skaitytiSveika, [18](#)
- spausdintiStream, [18](#)
- spausdintiRezultatus, [19](#)
- io.h
 - ivestiRankiniu, [22](#)
 - nuskaitytiStudentus, [22](#)
 - skaitytiSveika, [22](#)
 - spausdintiRezultatus, [22](#)
- isvestiKategorijaFaila
 - testavimas.cpp, [34](#)
 - testavimas.h, [43](#)
- ivestiRankiniu
 - io.cpp, [18](#)
 - io.h, [22](#)
- main
 - main.cpp, [23](#)
- main.cpp
 - main, [23](#)
- n
 - Studentas, [14](#)
- nd
 - Studentas, [14](#)
- nuskaitytiFaila
 - testavimas.cpp, [35](#)
 - testavimas.h, [43](#)
- nuskaitytiFailoT
 - testavimas.h, [43](#)
- nuskaitytiStudentus
 - io.cpp, [18](#)
 - io.h, [22](#)
- operator<<
 - studentas.cpp, [28](#)
 - studentas.h, [30](#)
- operator>>
 - studentas.cpp, [28](#)
 - studentas.h, [30](#)
- operator=
 - Studentas, [14](#)
- pasirinktiRusiavima
 - studentas_utils.cpp, [31](#)
 - studentas_utils.h, [33](#)
- pavarde
 - Zmogus, [15](#)
- rusiuotiPagalGalutini
 - testavimas.h, [43](#)
- rusiuotiStudentus

- studentas_utils.cpp, 31
- studentas_utils.h, 33
- skaiciavimas.cpp
 - skaiciuotiGalutini, 25
 - skaiciuotiMediana, 25
 - skaiciuotiVidurki, 26
- skaiciavimas.h
 - skaiciuotiGalutini, 27
 - skaiciuotiMediana, 27
 - skaiciuotiVidurki, 27
- skaiciuotiGalutini
 - skaiciavimas.cpp, 25
 - skaiciavimas.h, 27
- skaiciuotiMediana
 - skaiciavimas.cpp, 25
 - skaiciavimas.h, 27
- skaiciuotiVidurki
 - skaiciavimas.cpp, 26
 - skaiciavimas.h, 27
- skaitytiSveika
 - io.cpp, 18
 - io.h, 22
- spausdintiStream
 - io.cpp, 18
- spausdintiRezultatus
 - io.cpp, 19
 - io.h, 22
- strategija1
 - testavimas.h, 44
- strategija2
 - testavimas.h, 44
- strategija3
 - testavimas.h, 44
- Studentas, 12
 - ~Studentas, 13
 - egzaminas, 14
 - n, 14
 - nd, 14
 - operator=, 14
 - Studentas, 13
- studentas.cpp
 - operator<<, 28
 - operator>>, 28
- studentas.h
 - operator<<, 30
 - operator>>, 30
- studentas_utils.cpp
 - generuotiPazymius, 31
 - generuotiVarda, 31
 - pasirinktiRusiavima, 31
 - rusiuotiStudentus, 31
- studentas_utils.h
 - generuotiPazymius, 33
 - generuotiVarda, 33
 - pasirinktiRusiavima, 33
 - rusiuotiStudentus, 33
- testavimas.cpp
 - generuotiFaila, 34
 - isvestiKategorijaIFaila, 34
 - nuskaitytiFailo, 35
 - testuotiKlase, 35
 - tyrimas1_failuKurimas, 35
 - tyrimasKontaineriu, 35
 - tyrimasStrategiju, 35
- testavimas.h
 - apskaiciuotiGalutiniBala, 42
 - benchmarkKontaineris, 42
 - benchmarkStrategijos, 42
 - generuotiFaila, 43
 - isvestiKategorijaIFaila, 43
 - nuskaitytiFailo, 43
 - nuskaitytiFailoT, 43
 - rusiuotiPagalGalutini, 43
 - strategija1, 44
 - strategija2, 44
 - strategija3, 44
 - testuotiKlase, 44
 - tyrimas1_failuKurimas, 44
 - tyrimasKontaineriu, 44
 - tyrimasStrategiju, 45
- testuotiKlase
 - testavimas.cpp, 35
 - testavimas.h, 44
- tyrimas1_failuKurimas
 - testavimas.cpp, 35
 - testavimas.h, 44
- tyrimasKontaineriu
 - testavimas.cpp, 35
 - testavimas.h, 44
- tyrimasStrategiju
 - testavimas.cpp, 35
 - testavimas.h, 45
- vardas
 - Zmogus, 15
- Zmogus, 15
 - ~Zmogus, 15
 - pavarde, 15
 - vardas, 15
 - Zmogus, 15